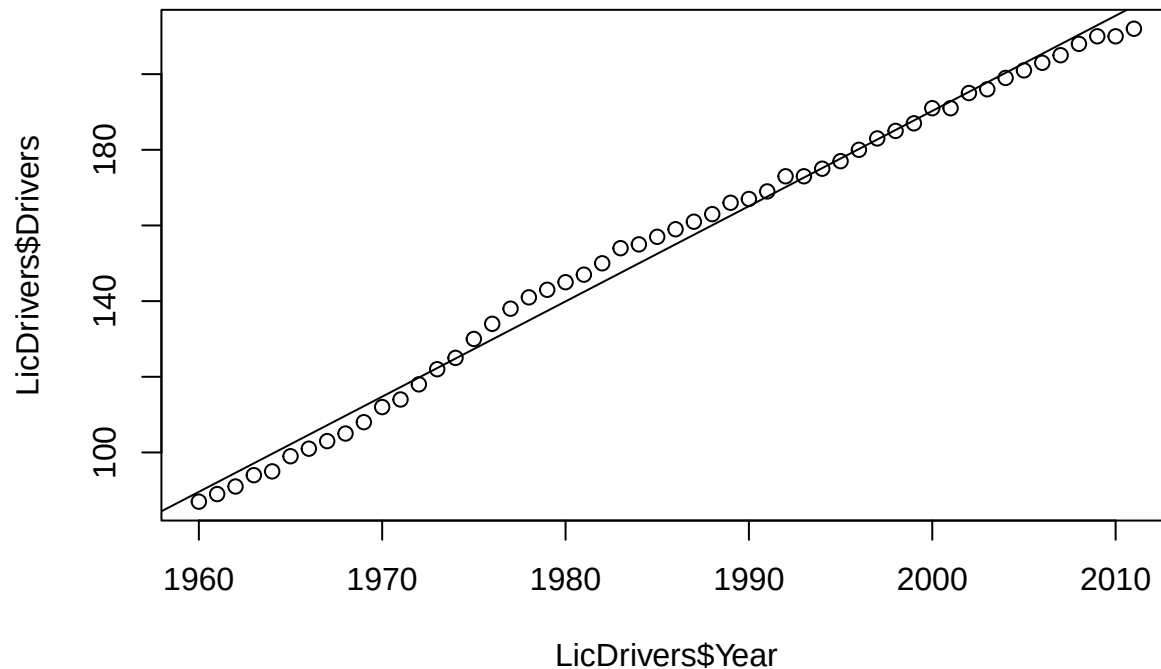


Checking Regression Model Assumptions

```
LicDrivers <- read.csv("/cloud/project/Data/LicensedDrivers.csv")
str(LicDrivers)
```

```
## 'data.frame': 54 obs. of 5 variables:
## $ Year      : num  1960 1961 1962 1963 1964 ...
## $ Population: int   180 183 186 188 191 194 196 197 199 201 ...
## $ Drivers   : int   87 89 91 94 95 99 101 103 105 108 ...
## $ Vehicles  : int   74 76 79 83 86 90 94 97 101 105 ...
## $ X         : logi  NA NA NA NA NA NA NA ...
```

```
LModel = lm(Drivers~Year,data = LicDrivers)
plot(LicDrivers$Year,LicDrivers$Drivers)
abline(LModel)
```



As we have seen before, there's a positive association between a year and the number of licensed drivers, and the regression line fits the data very well. Now, however, is the time to clarify the meaning of the values contained in the model.

To look at them, let's see a brief summary using the `summary()` function:

```
summary(LModel)
```

```
##
## Call:
## lm(formula = Drivers ~ Year, data = LicDrivers)
##
## Residuals:
```

```
##      Min      1Q  Median      3Q      Max
## -5.9601 -2.8996 -0.3038  3.0582  6.5240
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -4.844e+03  6.445e+01  -75.17  <2e-16 ***
## Year        2.517e+00  3.246e-02   77.56  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.513 on 50 degrees of freedom
## (2 observations deleted due to missingness)
## Multiple R-squared:  0.9918, Adjusted R-squared:  0.9916
## F-statistic: 6015 on 1 and 50 DF,  p-value: < 2.2e-16
```

The results show the linear regression model as a formula and following a series of residuals statistics. Now, go to the estimated coefficients, where there is one row for each of the terms included in the model, and the following columns:

Estimate: The estimated coefficient value for each corresponding term in the model. There are two rows: the first is the intercept and the second is the slope.

Std.Error: The standard error of the estimate. The average amount that the coefficient approximates varies from the actual average value of our response variable and is measured by the coefficient standard error. Ideally a lower number relative to its coefficients (as in our case).

t value: The t-statistic for each coefficient, for testing the null hypothesis that the corresponding coefficient is zero against the alternative that it is different from zero, given the other predictors in the model. This is a measure of how many standard deviations our coefficient estimate is far away from zero. This value is required to be far away from zero since this would indicate that the null hypothesis could be rejected—that is, it can be declared that there exists a relationship between speed and distance exists. In our case, the t-statistic values are relatively far away from zero and are large with respect to the standard error, which could suggest that a relationship exists. In general, t-values are also used to compute p-values.

Pr(>|t|): The p-value for the F-statistic of the hypotheses tests if the corresponding coefficient is equal to zero or not. In our example, the p-value of the F-statistic for Year is lower than 0.05, so this term is significant at the five percent significance level, given the other terms in the model.

T-test is a parametric test with the purpose of verifying whether the average value of a distribution differs significantly from a certain reference value.

In addition, we see the following information about the model we have created:

Residual standard error: A measure of the quality of a linear regression fit.

Multiple R-squared: The proportion of total sum of squares explained by the model. It represents the coefficient of determination. In our example, the R-squared value suggests that the model explains approximately 99 percent of the variability in the LicensedDrivers response variable.

Adjusted R-squared: This is a modified version of R-squared that has been adjusted for the number of predictors in the model.

F-statistic: The test statistic for the F-test on the regression model. It tests for a significant linear regression relationship between the response variable and the predictor variables.

p-value: The p-value for the F-test on the model. In our example, the model is significant with a p-value of 2.2e-16.

Let's look at the result of the last typed R command. Two values are highlighted on the others: R-squared and p-value. For the first, a very high value was calculated, equal to 0.992; it means that there is a strong variation in the response variable, explained by the predictor.

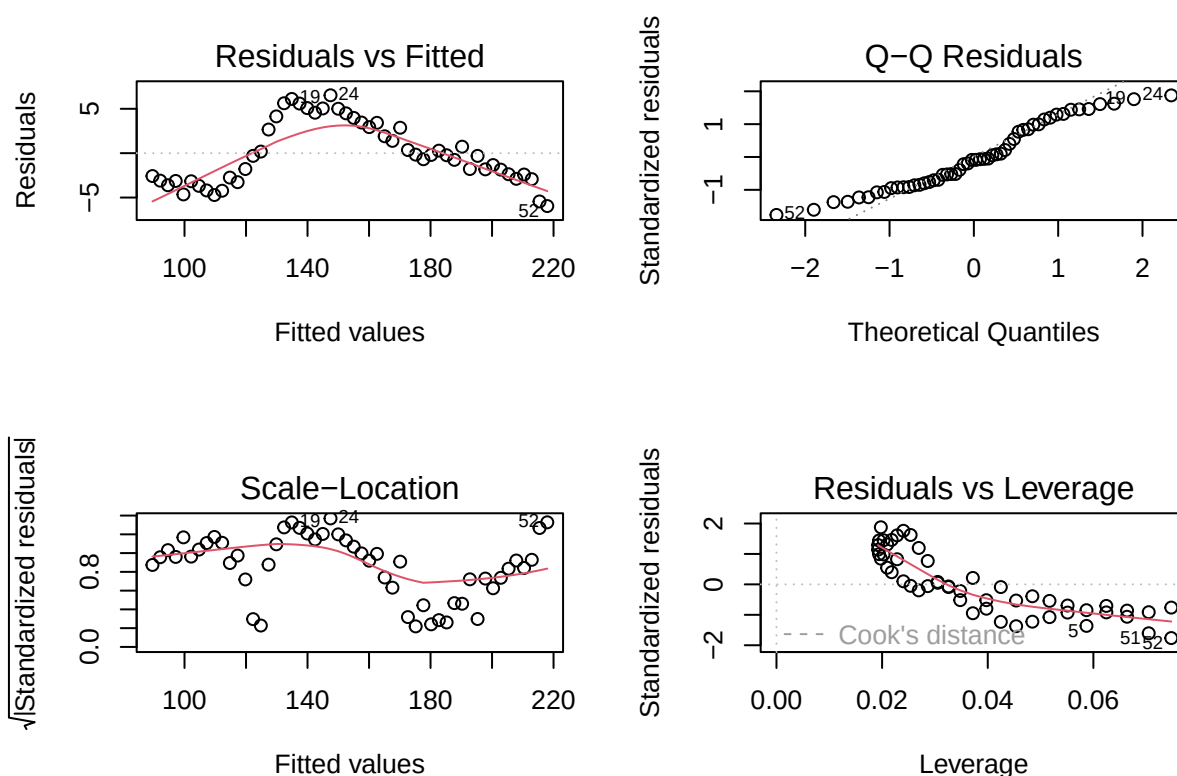
Diagnostic Plots

We have already looked at the residuals, but our analysis was superficial; it is necessary to deepen the subject. We will do it through integrated diagnostic diagrams for the linear regression analysis in R. It is enough to use the plot function with the linear regression model as the argument. Six plots are currently available:

- A plot of residuals against fitted values
- A normal q-q plot
- A scale-location plot of $\sqrt{|\text{residuals}|}$ against fitted values
- A plot of Cook's distances versus row labels
- A plot of residuals against leverages
- A plot of Cook's distances against leverage/(1-leverage)

By default, the first three and five are provided, and we will analyze them one by one. To select a single plot, use the which argument followed by the number (1-6). Alternatively, we can plot the results using the following command:

```
par(mfrow=c(2,2))
plot(LModel)
```

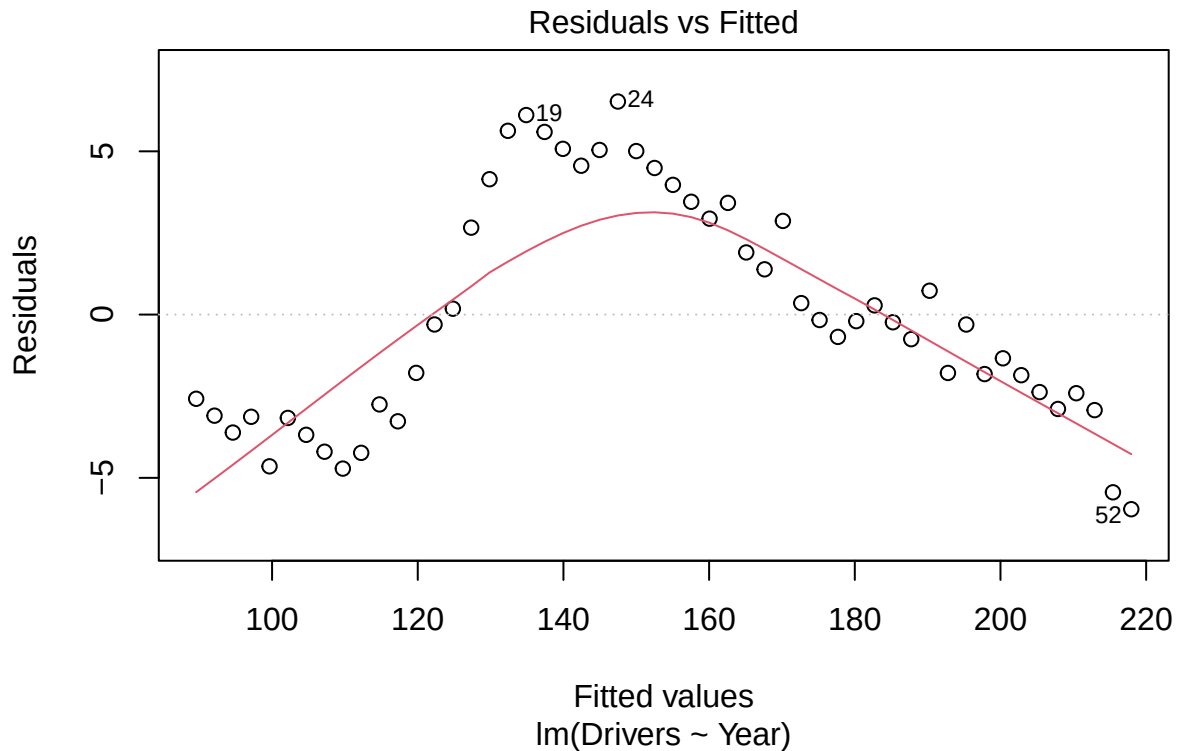


To return to a single graph per window, use `par(mfrow=c(1,1))`. Let's start with the plot that shows residuals against fitted values. This plot shows if residuals have nonlinear patterns. There could exist a nonlinear relationship between predictor and outcome variables and the pattern could show up in this plot if the nonlinear relationship is not captured by the model. A good indication that our model fits very well is if there are equally spread residuals around a horizontal line without distinct patterns.

The residuals should be distributed randomly around the horizontal line representing a residual error of zero, such that there should be no clear trend in the distribution of the points. Let's look at residual plots from our model:

The following figure shows the Residuals versus Fitted values:

```
par(mfrow=c(1,1))
plot(LModel,which=1)
```



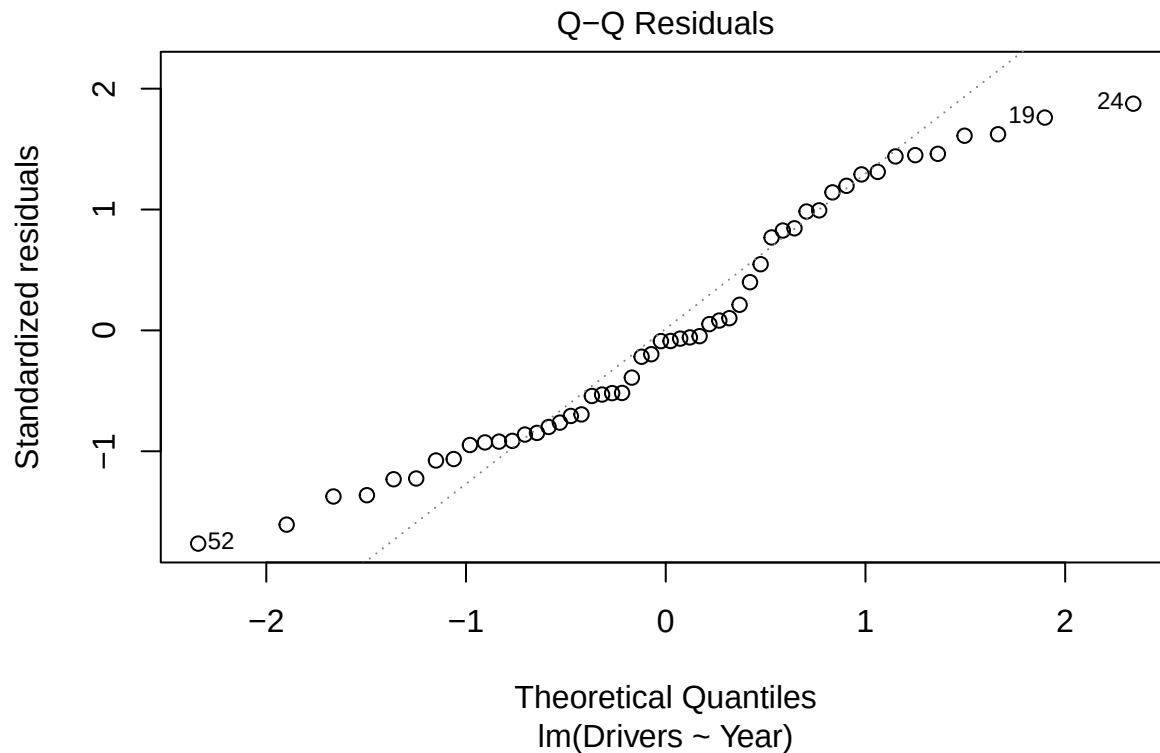
The dotted line at Residuals is equal to zero indicates our fit line. Any point on the fit line obviously has zero residuals. Previous points have positive residuals and the following points have negative residuals. In particular, some observations are highlighted by indicating the number. These observations are the most problematic. In this case, observations number 19, 24, and 52 are highlighted.

Remember a residual is a measure of how well a regression line fits an individual data point. Therefore, if the residuals appeared to behave randomly, it indicates that the model fits the data well. However, if the residuals appear to show a systematic pattern, we can clearly say that the model fits the data poorly.

In our case, we can see that our residuals have a curved pattern (red line). The fitted line doesn't describe how the response behaves when the input changes, since the relationship is nonlinear. This could mean that we may get a better result if we try a model with a quadratic term included. This is an unexpected result, as the R-squared value was very close to one. All this confirms that there is still much to be observed. We will explore this point further in the following chapters, by actually trying this to see if it helps.

Let's look at the second diagnostic plot: normal q-q plot. This plot shows if residuals are normally distributed. It's good if residuals are lined well on the straight dashed line:

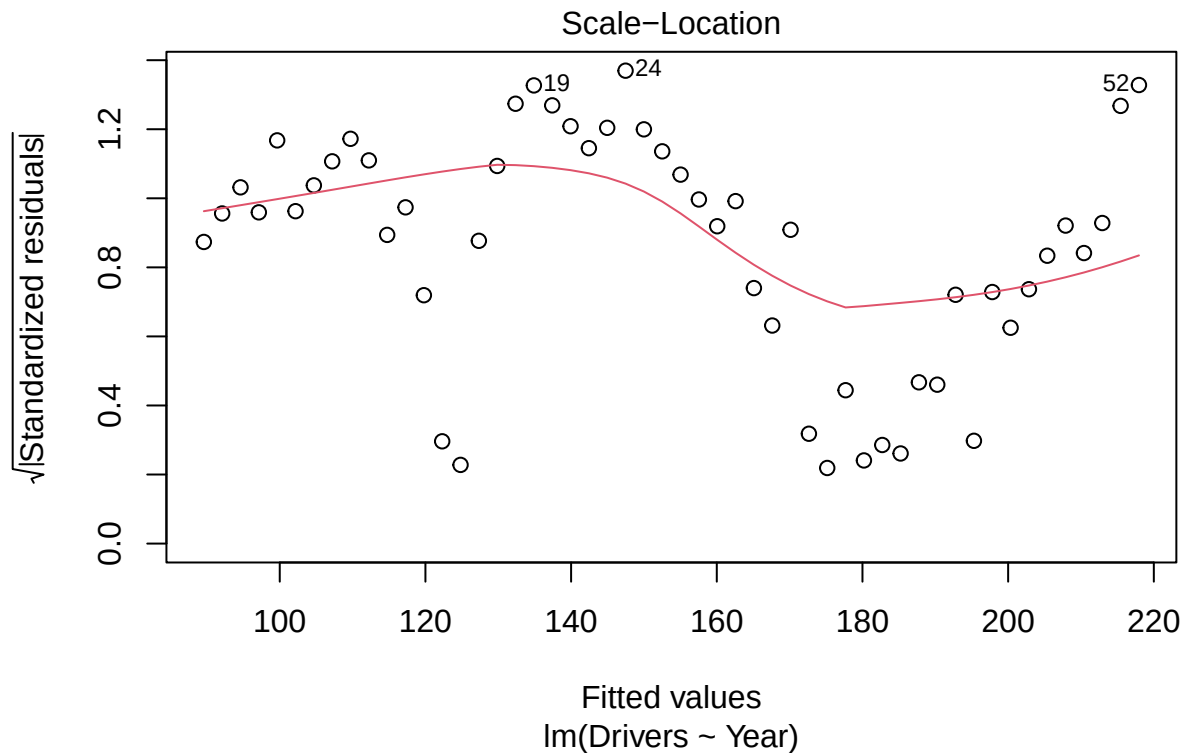
```
plot(LModel,which=2)
```



We are looking for if the errors are approximately normally distributed, which is what the q-q plot allows you to see. Analyzing the figure, it can be noted that the residuals do not follow a straight line well. Once again, there are areas where the residuals are located over the dashed lines and areas in which they are located as follows.

Let's look at the third diagnostic plot: scale-location plot, also called a spread-location plot. This plot shows if residuals are spread equally along the ranges of predictors. This is how you can check the assumption of equal variance (homoscedasticity). Indeed, we can also check this assumption with the first diagnostic plot. It's good if you see a horizontal line with equally (randomly) spread points:

```
plot(LModel,which=3)
```



The scale-location plot suggests the spread of points across the predicted values range. Homoscedasticity is one of the assumptions for regression, such that variance should be reasonably equal across the predictor range. It is ideal to have a horizontal red line that would indicate the residuals have uniform variance across the range.

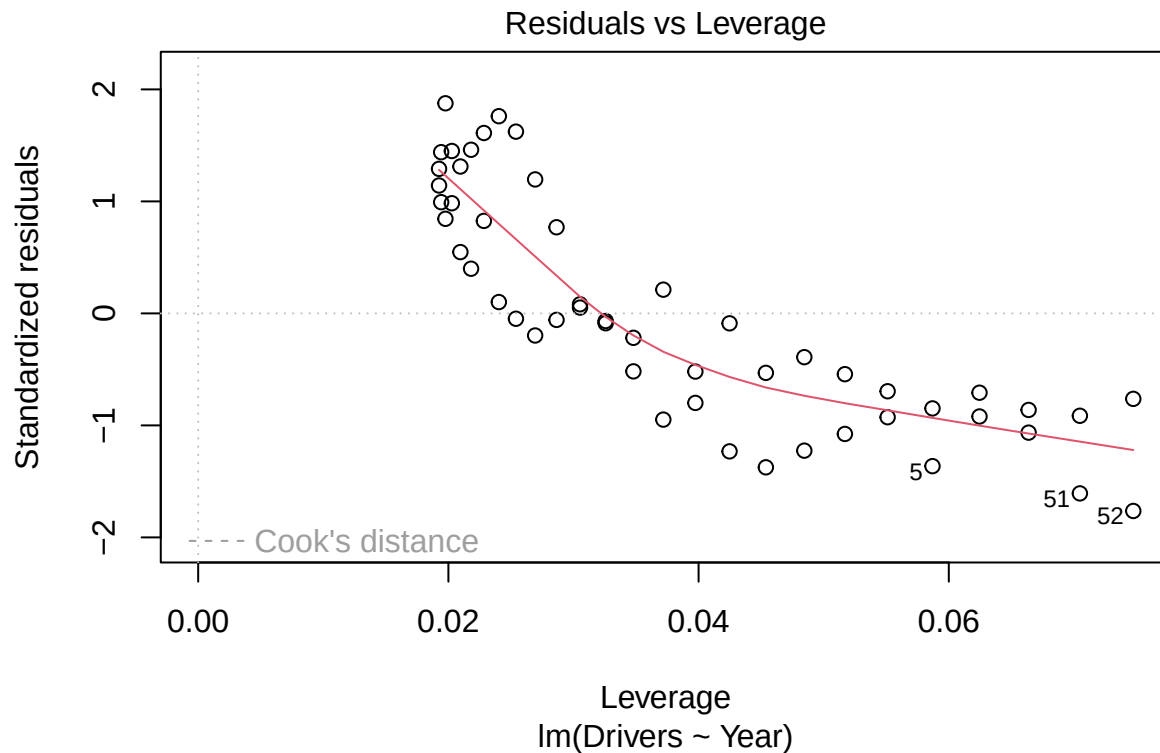
Even in this case, there must be no obvious trend in this plot. Analyzing the figure, it can be noted that the residuals are not equally distributed. Because the residuals focus on some areas at the expense of others, the red smooth line is not horizontal and shows a steep angle.

The last diagnostic plot that we have to analyze is a plot of residuals against leverages. This plot helps us to determine any influential cases, if present. All outliers are not necessarily influential in linear regression analysis. Data might not necessarily be influential in determining a regression line even though the data has extreme values. This means that whether we decide to include or exclude these data from the analysis the results wouldn't be much different.

Before drawing the plot, we must know what influence and leverage are. The amount of change in the predicted scores if an observation is excluded determines the influence of the observation. Cook's distance is a pretty good measure of the influence of an observation. The leverage of an observation is based on how much the observation's value on the predictor variable differs from the mean of the predictor variable. The more the leverage of an observation, the greater potential that point has in terms of influence.

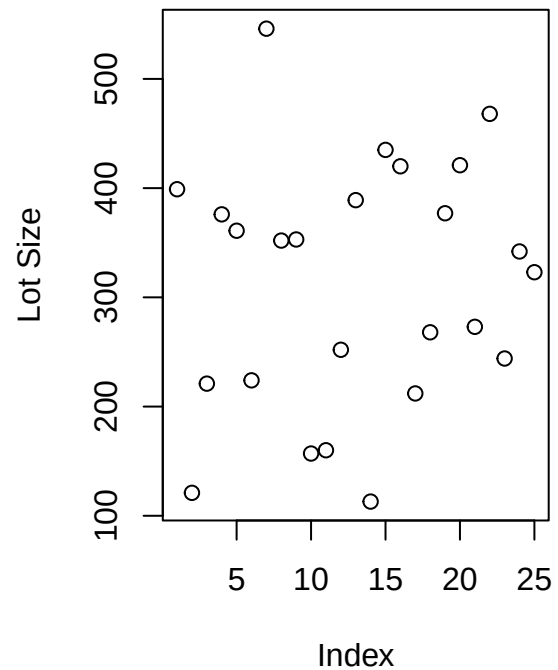
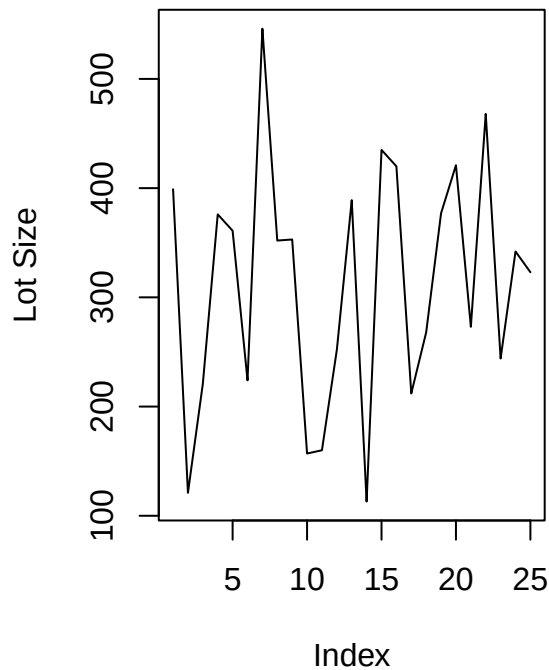
These time patterns are not relevant unlike the other plots. We watch out for outlying values at the upper-right or lower-right corner. Those are the spots where cases can be influential against a regression line. If we find cases outside of a dashed line, these lines are referred to as Cook's distance. The cases are influential to the regression results, when cases are outside of Cook's distance. If those cases are excluded the regression results will be altered. Let's draw this plot:

```
plot(LModel,which=5)
```



In this plot, Cook's distance is represented by the dotted red lines and the ones outside the dotted line at the top-right or bottom-right corner are the ones that are of interest to us. We say that the observation has high leverage or that the potential for influencing our model is higher if any point falls in that region and if we exclude that point. As we can notice there are points in the bottom right corner. We will take a look at this in the following sections.

```
toluca_data <- read.csv("/cloud/project/Data/toluca_data.csv")
#we want two graphs on the same page
par(mfrow=c(1,2))
#type="l" will draw line
plot(toluca_data$workhrs, type="l", ylab="Lot Size")
#type="p" is the default option, will give you the scatter plot
plot(toluca_data$workhrs, type="p", ylab="Lot Size")
```

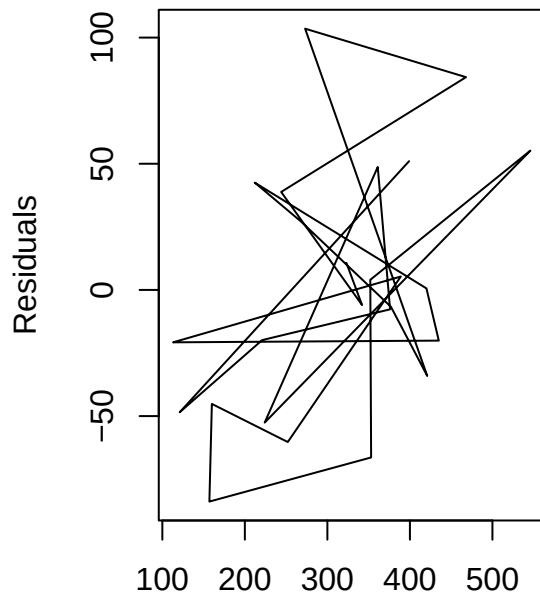


```
# fitting the regression model
toluca.reg <- lm(workhrs ~ lotsize,data=toluca_data)
# getting the summary regression output:
summary(toluca.reg)
```

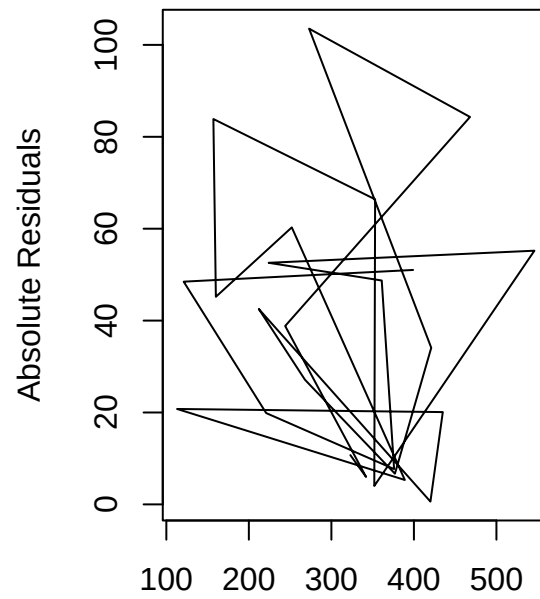
```
##
## Call:
## lm(formula = workhrs ~ lotsize, data = toluca_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -83.876 -34.088  -5.982  38.826 103.528
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   62.366     26.177   2.382  0.0259 *
## lotsize        3.570      0.347  10.290 4.45e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 48.82 on 23 degrees of freedom
## Multiple R-squared:  0.8215, Adjusted R-squared:  0.8138
## F-statistic: 105.9 on 1 and 23 DF,  p-value: 4.449e-10
```

```
#error vs. Xi plot
ei<-toluca.reg$residuals
plot(toluca_data$workhrs,ei, type="l", xlab="Lot Size",ylab="Residuals")

#abs(error) vs. Xi plot
plot(toluca_data$workhrs,abs(ei), type="l", xlab="Lot Size",ylab="Absolute Residuals")
```



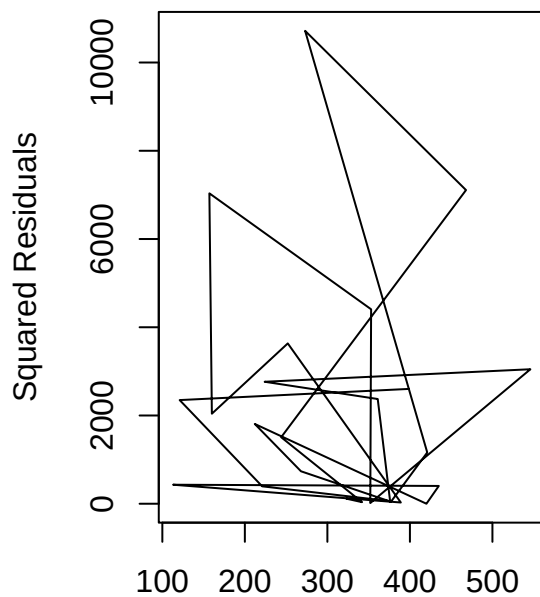
Lot Size



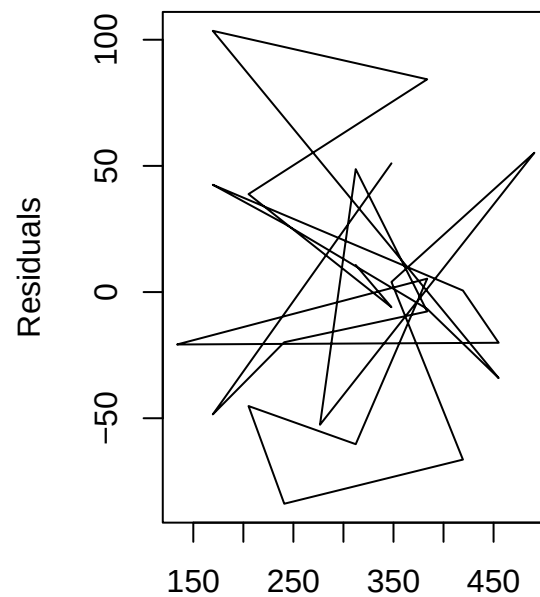
Lot Size

```
# error^2 vs. Xi plot
plot(toluca_data$workhrs,ei^2, type="l", xlab="Lot Size",ylab="Squared Residuals")

# ei. vs predicted
plot(toluca.reg$fitted.values,ei, type="l", xlab="Lot Size",ylab="Residuals")
```



Lot Size



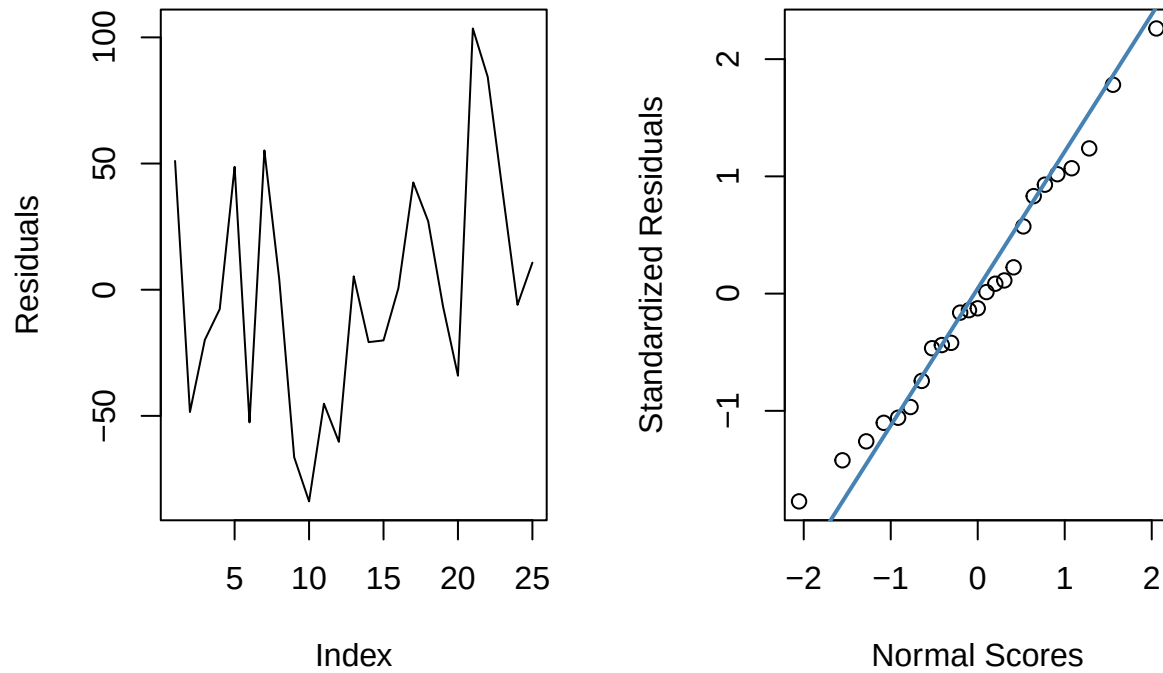
Lot Size

```
# ei. vs time sequence
plot(ei, type="l",ylab="Residuals")

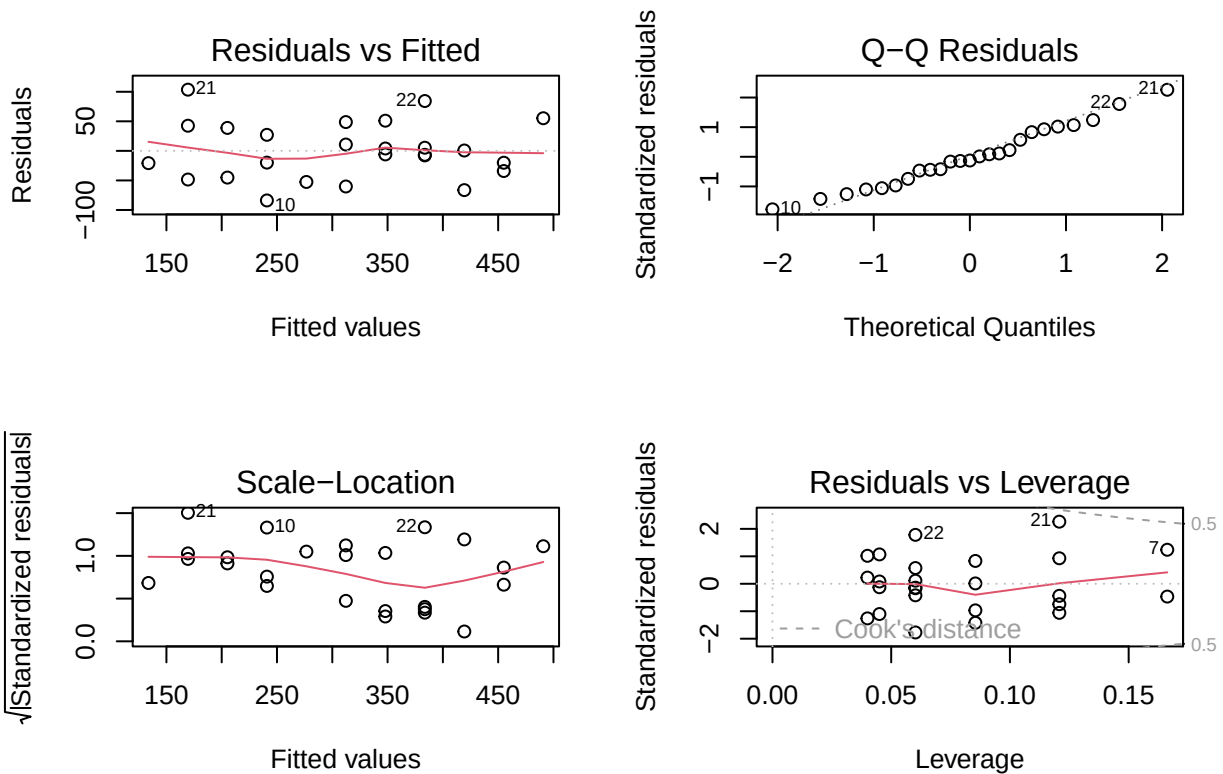
# QQ PLOT (aka normal probability plot)
```

```
error.std = rstandard(toluca.reg)
qqnorm(error.std,ylab="Standardized Residuals",xlab="Normal Scores",main="Toluca Data")
qqline(error.std, col = "steelblue", lwd = 2)
```

Toluca Data



```
#alternatively, you can QQ plot
par(mfrow=c(2,2))
plot(toluca.reg)
```



```
##Correlation Test for Normality
```

H_o : Errors are normally distributed

H_a : Errors are Not normally distributed

```
shapiro.test(toluca.reg$residuals)
```

```
##
```

```
## Shapiro-Wilk normality test
```

```
##
```

```
## data: toluca.reg$residuals
```

```
## W = 0.9789, p-value = 0.8626
```

P value is 0.86 ($P\text{-Value} > -0.05$). Accept H_o , Errors are normally distributed.

Brown-Forsythe Test

H_o : Error variances are constant

H_a : Error variances are not constant

```
Dataset_1.20 <- read.csv("/cloud/project/Data/Dataset_1_20.csv", header=FALSE)
```

```
f3.4<-lm(Dataset_1.20$V1 ~ Dataset_1.20$V2)
```

```
summary(f3.4)
```

```
##
```

```
## Call:
```

```
## lm(formula = Dataset_1.20$V1 ~ Dataset_1.20$V2)
```

```
##
```

```
## Residuals:
```

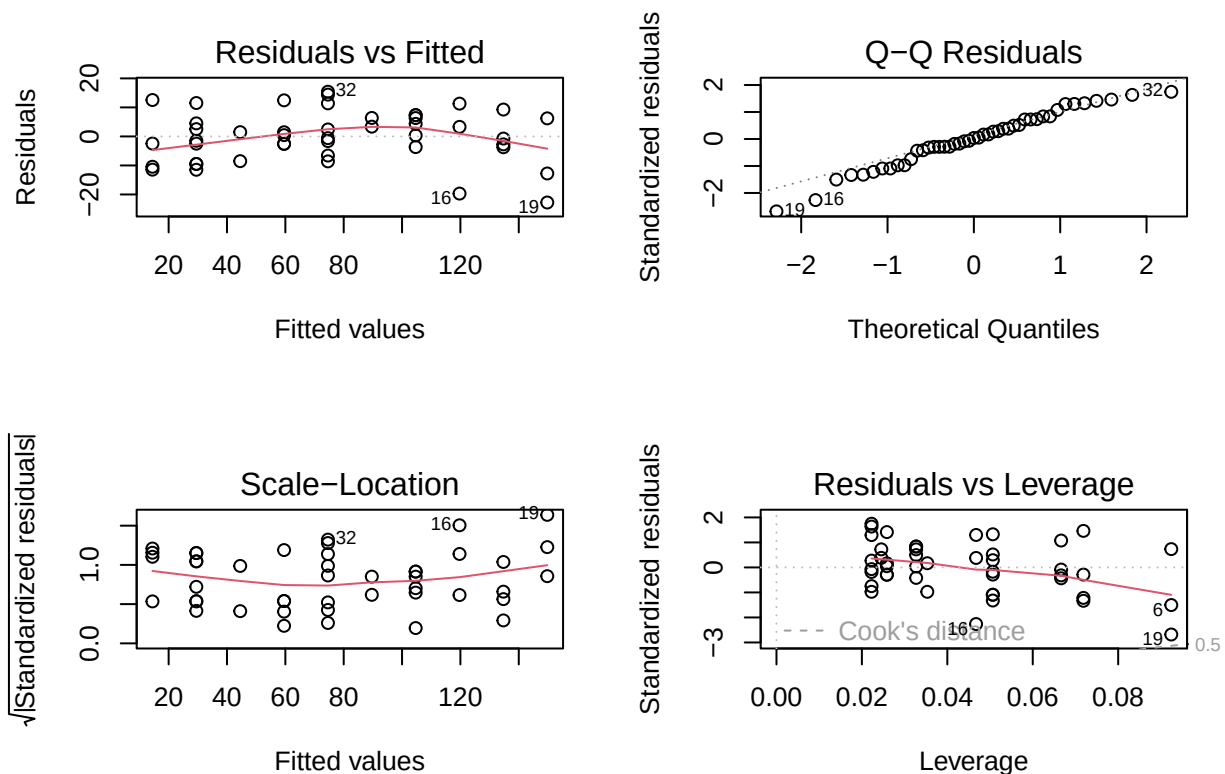
```
##      Min       1Q   Median       3Q      Max
```

```
## -22.7723  -3.7371   0.3334   6.3334  15.4039
```

```
##
```

```
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -0.5802     2.8039  -0.207   0.837
## Dataset_1.20$V2 15.0352     0.4831  31.123 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8.914 on 43 degrees of freedom
## Multiple R-squared:  0.9575, Adjusted R-squared:  0.9565
## F-statistic: 968.7 on 1 and 43 DF,  p-value: < 2.2e-16
```

```
par(mfrow=c(2,2))
plot(f3.4)
```



```
ei<-f3.4$residuals
summary(Dataset_1.20$V2)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.000  2.000   5.000   5.111  7.000   10.000
```

#We will separate the data based on the median value, we first need to create a data frame with X,Y and

```
pop1.20<-data.frame(cbind(Dataset_1.20$V1, Dataset_1.20$V2,ei))
p1<-pop1.20[pop1.20[,2]< 5,]
p2<-pop1.20[pop1.20[,2]>=5,]
```

#get the medians for each sample

```
M1<-summary(p1[,3])[3]
```

```
M2<-summary(p2[,3])[3]
```

#get the n for each sample

```
N1<-dim(p1)[1]
```

```

N2<-dim(p2)[1]

#calculate dij = abs(error - median.error)
d1<-abs(p1[,3]-M1)
d2<-abs(p2[,3]-M2)

#calculate the pooled variance
s2<-sqrt((var(d1)*(N1-1)+var(d2)*(N2-1))/(N1+N2-2))

#calcualte the t statistics and p value
Den<- s2*sqrt(1/N1+1/N2)
Num<- mean(d1)-mean(d2)

T= Num/Den
1-pt(T,N1+N2-2)

```

```
## [1] 0.7526557
```

P value is 0.75 ($P\text{-Value} > -0.05$). Accept H_o , Errors are normally distributed. Error variances are constant

```
library(car)
```

```
## Loading required package: carData
```

```
group=I(pop1.20[,2]< 5)*1
leveneTest(ei, group, center=median)
```

```
## Warning in leveneTest.default(ei, group, center = median): group coerced to
## factor.
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##      Df F value Pr(>F)
## group 1  0.4744 0.4947
##      43
```

Using the built in function, which based on F test, variance calculations are different.

```
library(car)
group=I(pop1.20[,2]< 5)*1
leveneTest(Dataset_1.20$V1, group, center=median)
```

```
## Warning in leveneTest.default(Dataset_1.20$V1, group, center = median): group
## coerced to factor.
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##      Df F value Pr(>F)
## group 1  1.5462 0.2204
##      43
```

P value is 0.22 ($P\text{-Value} > -0.05$). Accept H_o , Error variances are constant

Breusch-Pagan Test

```

#using the library
library(olsrr)

```

```
##
## Attaching package: 'olsrr'
```

```
## The following object is masked from 'package:datasets':
##
##     rivers
```

```
ols_test_breusch_pagan(f3.4)
```

```
##
## Breusch Pagan Test for Heteroskedasticity
## -----
## Ho: the variance is constant
## Ha: the variance is not constant
##
##              Data
## -----
## Response : Dataset_1.20$V1
## Variables: fitted values of Dataset_1.20$V1
##
##      Test Summary
## -----
## DF          =      1
## Chi2         =     1.31468
## Prob > Chi2  =     0.2515491
```

Performing Manually

H_o : Error variances are constant

H_a : Error variances are not constant

#from the new model, get SSR.S

```
ei<-f3.4$residuals
ei2<-ei^2
g<-lm(ei2~Dataset_1.20$V2)
summary(g)
```

```
##
## Call:
## lm(formula = ei2 ~ Dataset_1.20$V2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -101.32  -69.40  -41.29   54.59  410.04
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    41.818     32.732   1.278   0.208
## Dataset_1.20$V2    6.672      5.639   1.183   0.243
##
## Residual standard error: 104.1 on 43 degrees of freedom
## Multiple R-squared:  0.03153,    Adjusted R-squared:  0.009004
## F-statistic:  1.4 on 1 and 43 DF,  p-value: 0.2433
```

```
anova(g)
```

```
## Analysis of Variance Table
##
## Response: ei2
##              Df Sum Sq Mean Sq F value Pr(>F)
```

```
## Dataset_1.20$V2  1  15155   15155   1.3998 0.2433
## Residuals       43 465556   10827

SSR.S= 15155
#from the original model get SSE
anova(f3.4)

## Analysis of Variance Table
##
## Response: Dataset_1.20$V1
##           Df Sum Sq Mean Sq F value    Pr(>F)
## Dataset_1.20$V2  1  76960   76960  968.66 < 2.2e-16 ***
## Residuals       43   3416     79
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

SSE = 3416

#n is 45
n=45
Chi_Square=(SSR.S/2) / ((SSE/n)^2)
1-pchisq(Chi_Square,1)

## [1] 0.2514971

P value is 0.25 ( $P\text{-Value} > -0.05$ ). Accept  $H_o$ , Error variances are constant
```

Lack of Fit Test

H_o : There is no lack of fit H_a : There is lack of fit

```
#Library below can only be used for one independent variable
#install.packages("EnvStats")
library(EnvStats)
```

```
##
## Attaching package: 'EnvStats'

## The following object is masked from 'package:car':
##
##     qqPlot

## The following objects are masked from 'package:stats':
##
##     predict, predict.lm

## The following object is masked from 'package:base':
##
##     print.default

anovaPE(toluca.reg)

##           Df Sum Sq Mean Sq F value    Pr(>F)
## lotsize      1 252378   252378  94.0183 1.367e-07 ***
## Lack of Fit    9  17245     1916  0.7138  0.6893
## Pure Error    14  37581     2684
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#Alternative way of performing lack of fit
Reduced <- lm(workhrs ~ lotsize,data=toluca_data)
Full <- lm(workhrs ~ 0 + as.factor(lotsize),data = toluca_data)
anova(Reduced, Full)
```

```
## Analysis of Variance Table
##
## Model 1: workhrs ~ lotsize
## Model 2: workhrs ~ 0 + as.factor(lotsize)
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      23 54825
## 2      14 37581  9      17245 0.7138 0.6893
```

P value is 0.69 ($P\text{Value} > -0.05$). Accept H_o , There is no lack of fit.

Box Cox Plot

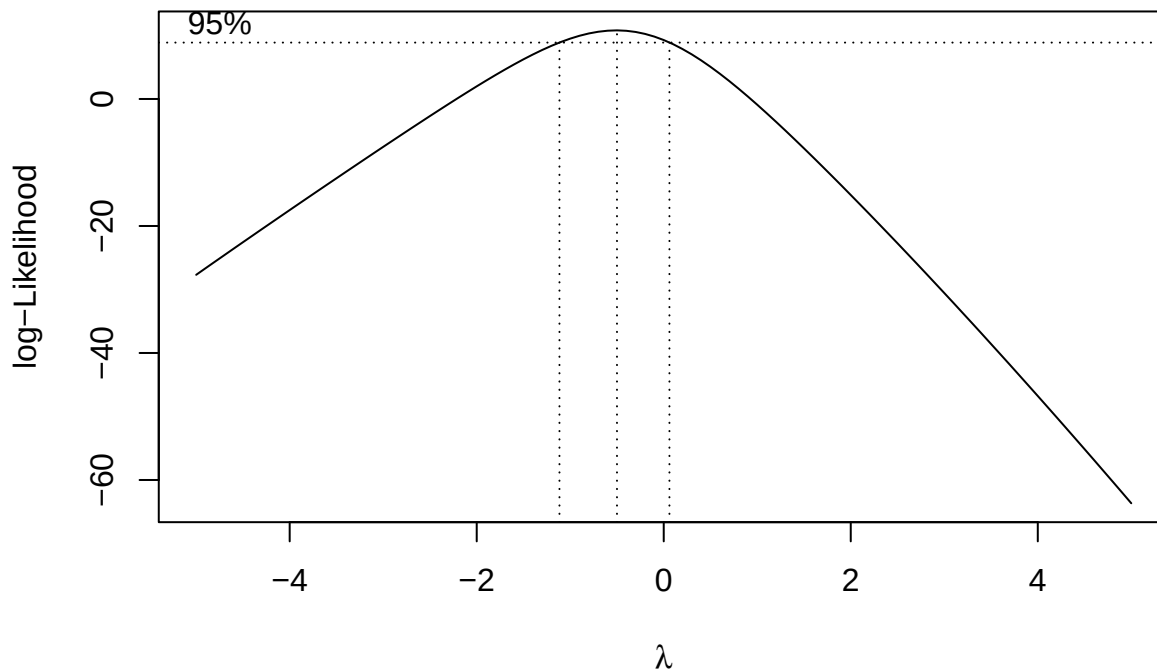
```
library(MASS)

##
## Attaching package: 'MASS'

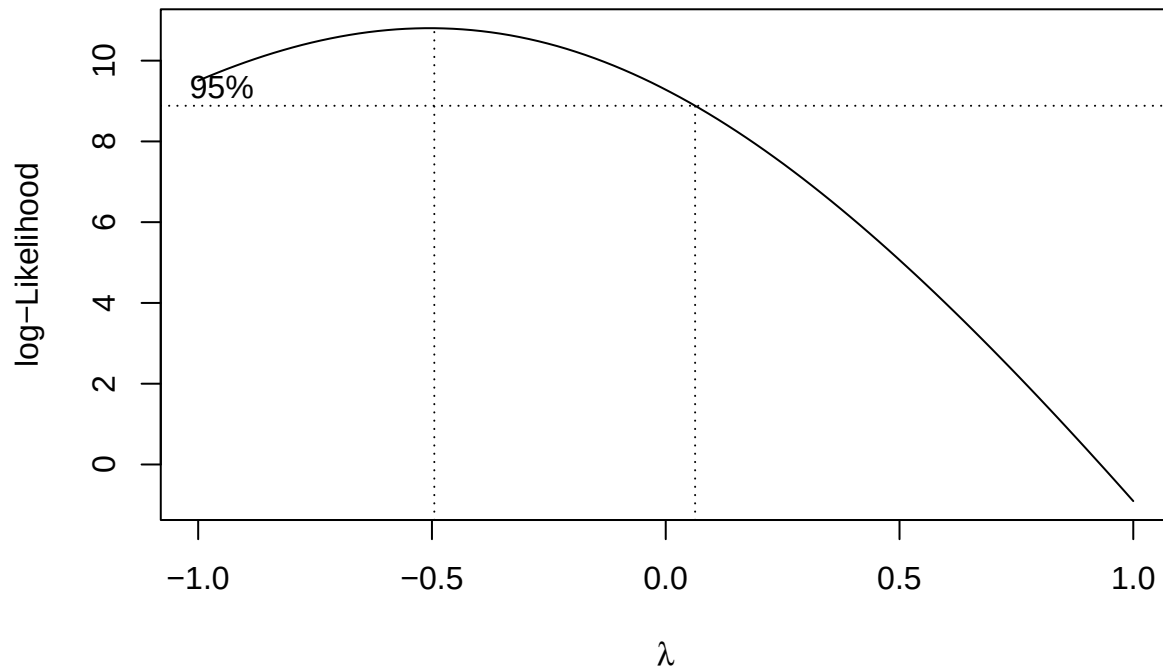
## The following object is masked from 'package:EnvStats':
##
##   boxcox

## The following object is masked from 'package:olsrr':
##
##   cement

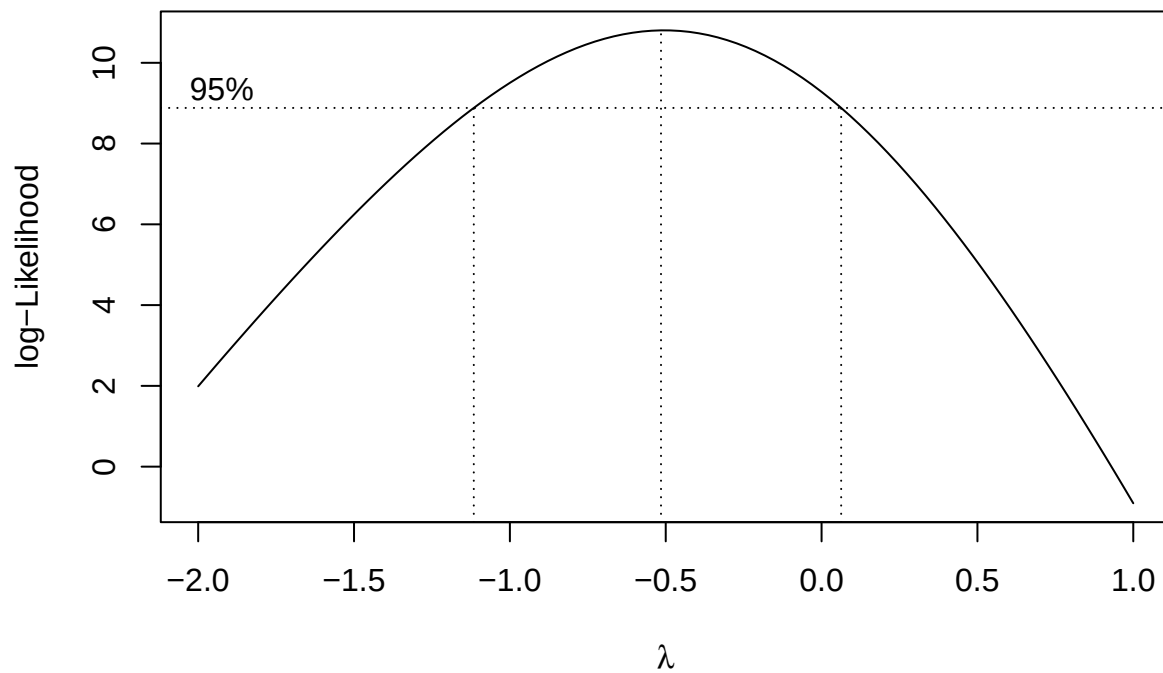
par(mfrow=c(1,1))
PlasmaLevel <- read.csv("/cloud/project/Data/PlasmaLevel.csv")
f<-lm(Y~X,data=PlasmaLevel)
boxcox(f,lambda=seq(-5,5,by=.1))
```



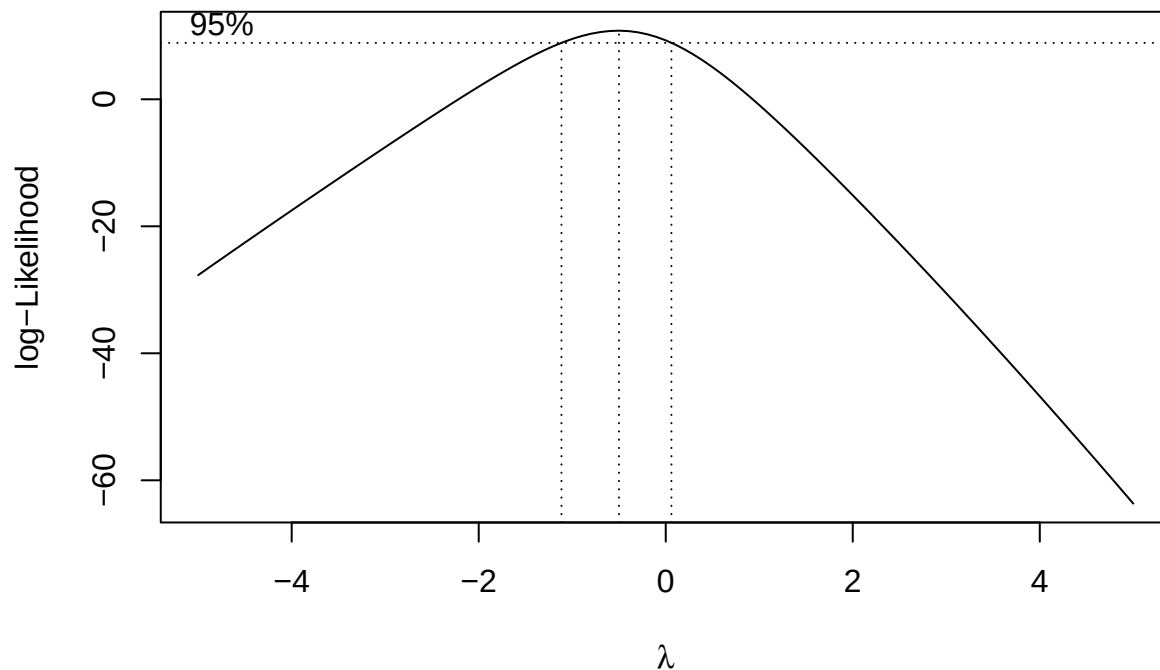
```
boxcox(f,lambda=seq(-1,1,by=.1))
```



```
boxcox(f,lambda=seq(-2,1,by=.05))
```



```
#actual value lambda  
a=boxcox(f,lambda=seq(-5,5,by=.1))
```



```
lamda=a$x[which.max(a$y)]
lamda
```

```
## [1] -0.5
```

```
f1<-lm(LogY~X,data=PlasmaLevel)
f2<-lm(Y^lamda~X,data=PlasmaLevel)
summary(f1)
```

```
##
## Call:
## lm(formula = LogY ~ X, data = PlasmaLevel)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.116974 -0.035356  0.002626  0.027908  0.168180
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.134820   0.021641   52.44 < 2e-16 ***
## X           -0.102282   0.008835  -11.58 4.51e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06247 on 23 degrees of freedom
## Multiple R-squared:  0.8535, Adjusted R-squared:  0.8472
## F-statistic: 134 on 1 and 23 DF, p-value: 4.506e-11
```

```
summary(f2)
```

```
##
## Call:
## lm(formula = Y^lamda ~ X, data = PlasmaLevel)
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.04492 -0.01257 -0.00388  0.01453  0.05287
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.268026    0.008033   33.36 < 2e-16 ***
## X            0.040062    0.003280   12.22 1.55e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02319 on 23 degrees of freedom
## Multiple R-squared:  0.8665, Adjusted R-squared:  0.8606
## F-statistic: 149.2 on 1 and 23 DF,  p-value: 1.548e-11
```

$\lambda = -0.5$ gives the maximum log-likelihood. 95% confidence interval for λ is roughly between $-1.1 \leq \lambda \leq 0.1$. Choosing $\lambda = 0$, means that log transformation should be used and it is more transparent and easy to explain. In other words, we will comprise on the power of the model little bit to get more transparent model.

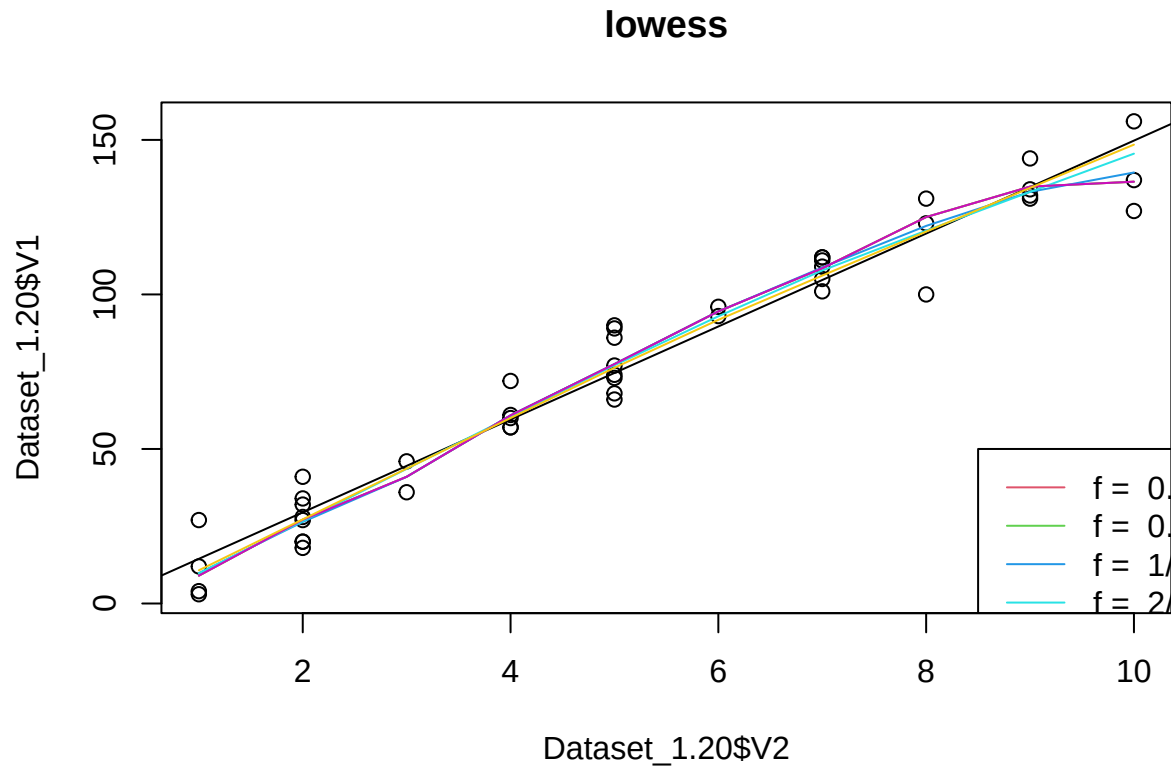
##Lowess Regression

The neighborhood is defined as the span, which is the fraction of the total points used to form neighborhoods.

A span 0.5 indicates that the closest half of the total data points is used as the neighborhood. Then lowess procedure uses the points in the neighborhood to generate a weighted least-squares estimate of the specific response.

- Base R smoother using weighted local regression
- Fast, simple, ideal for quick visualization
- Key parameter: f (smoothing span, 0–1)

```
par(mfrow=c(1,1))
f3.4<-lm(Dataset_1.20$V1 ~ Dataset_1.20$V2)
plot(Dataset_1.20$V1 ~ Dataset_1.20$V2, main = "lowess")
abline(f3.4,col=1)
lines(lowess(Dataset_1.20$V1 ~ Dataset_1.20$V2,f=0.01), col = 2)
lines(lowess(Dataset_1.20$V1 ~ Dataset_1.20$V2,f=0.1), col = 3)
lines(lowess(Dataset_1.20$V1 ~ Dataset_1.20$V2,f=1/3), col = 4)
lines(lowess(Dataset_1.20$V1 ~ Dataset_1.20$V2,f=2/3), col = 5)
lines(lowess(Dataset_1.20$V1 ~ Dataset_1.20$V2, f=.2), col = 6)
lines(lowess(Dataset_1.20$V1 ~ Dataset_1.20$V2, f=1), col = 7)
#legend(5, 120, c(paste("f = ", c("0.01","0.1","1/3","2/3", ".2","1"))), lty = 1, col = 2:7)
legend(8.5, 50, c(paste("f = ", c("0.01","0.1","1/3","2/3", ".2","1"))), lty = 1, col = 2:7)
```



f: The value for the smoother span. This gives the proportion of points in the plot which influence the smooth at each value. Larger values result in more smoothness.

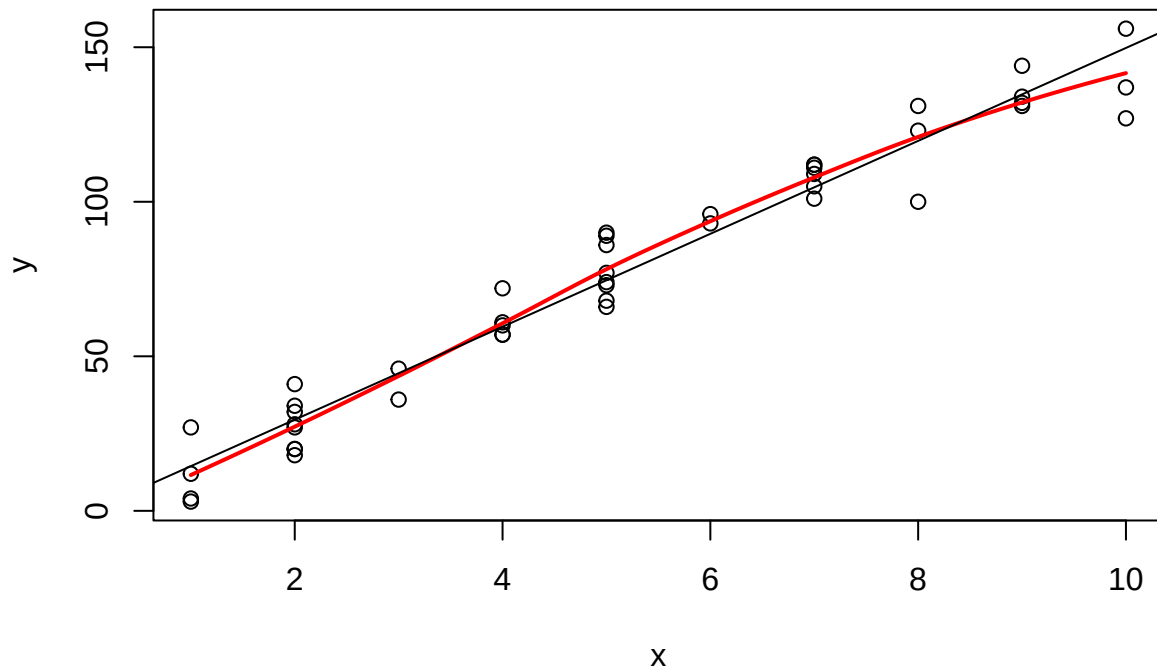
##LOESS Regression

- More flexible model with formula interface
- Supports multiple predictors and prediction
- Key parameters:
 - span (smoothing)
 - degree (1 = linear, 2 = quadratic)
 - family (“gaussian” or robust “symmetric”)

```
y=Dataset_1.20$V1
x=Dataset_1.20$V2
model <- loess(y ~ x, span = 0.75)

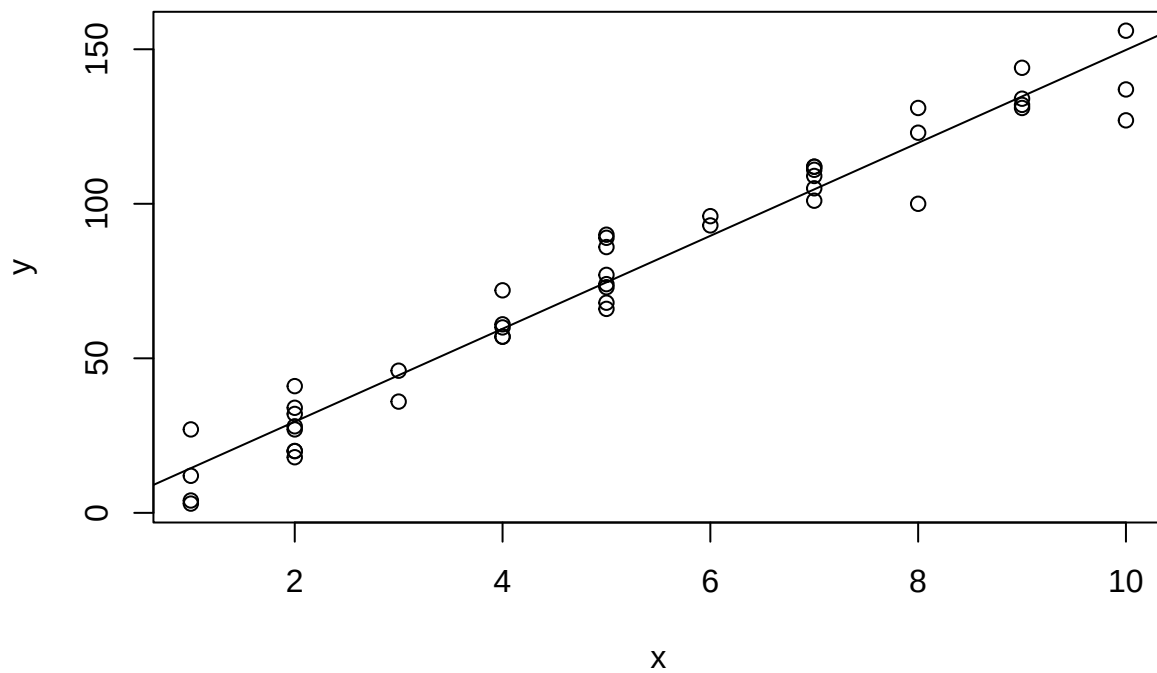
x_new <- seq(min(x), max(x), length.out = 200)
y_hat <- predict(model, newdata = data.frame(x = x_new))

plot(x, y)
lines(x_new, y_hat, col = "red", lwd = 2)
abline(f3.4)
```



```
plot(x, y, main="Linear Regression")
abline(f3.4)
```

Linear Regression



#compare LOESS and Linear Regression on Prostate Data

Performances are very similar.

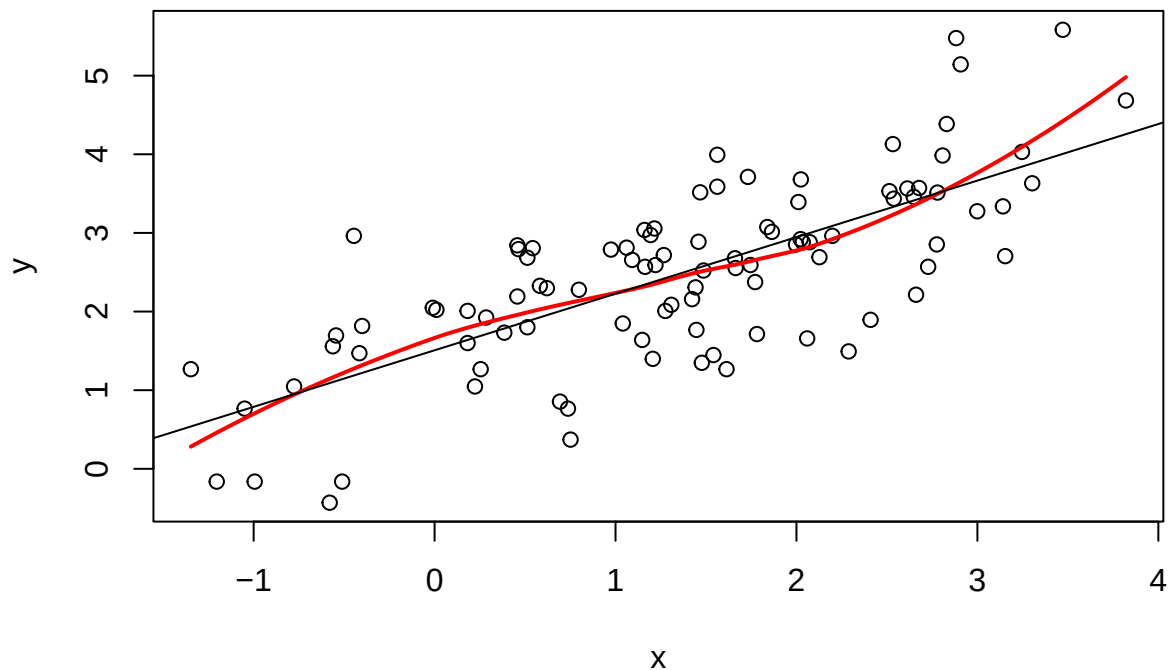
```
#loading Prostate Data
load(url("https://raw.githubusercontent.com/cran/faraway/master/data/prostate.rda"))
head(prostate)
```

```
##      lcavol lweight age      lbph svi      lcp gleason pgg45      lpsa
## 1 -0.5798185 2.7695 50 -1.386294 0 -1.38629      6      0 -0.43078
## 2 -0.9942523 3.3196 58 -1.386294 0 -1.38629      6      0 -0.16252
## 3 -0.5108256 2.6912 74 -1.386294 0 -1.38629      7     20 -0.16252
## 4 -1.2039728 3.2828 58 -1.386294 0 -1.38629      6      0 -0.16252
## 5  0.7514161 3.4324 62 -1.386294 0 -1.38629      6      0  0.37156
## 6 -1.0498221 3.2288 50 -1.386294 0 -1.38629      6      0  0.76547
```

```
y=prostate$lpsa
x=prostate$lcavol

#loess Regression
modelLoess <- loess(y ~ x, span = 0.75)

x_new <- seq(min(x), max(x), length.out = 200)
y_hat <- predict(modelLoess, newdata = data.frame(x = x_new))
#linear regression
f4<-lm(y~x)
plot(x, y)
lines(x_new, y_hat, col = "red", lwd = 2)
abline(f4)
```



```
#Calculate model performance Stats

library(caret)

## Loading required package: ggplot2
## Loading required package: lattice

#performance of LOESS
#get the predicted values
PredictedLoess<-predict(modelLoess, newdata = data.frame(x))
```

```

#create a data frame with 2 columns actual and predicted values
ModelDataLoess<-data.frame(obs = y, pred=PredictedLoess)
#calculate performance statistics
outModelDataLoess<-defaultSummary(ModelDataLoess)

#performance of Regression
#get the predicted values
PredictedReg<-predict(f4, newdata = data.frame(x))
#create a data frame with 2 columns actual and predicted values
ModelDataReg<-data.frame(obs = y, pred=PredictedReg)
#calculate performance statistics
outModelDataReg<-defaultSummary(ModelDataReg)

out<-rbind(outModelDataLoess,outModelDataReg)
dimnames(out)[[1]]<-c("LOESS","Reg")
out

```

```

##           RMSE  Rsquared      MAE
## LOESS 0.7651398 0.5560669 0.6072555
## Reg   0.7793386 0.5394319 0.6157369

```

##Correlation Test

$H_o: \rho=0$

$H_a: \rho \neq 0$

The Spearman's rank-order correlation is the nonparametric version of the Pearson product-moment correlation. Spearman's correlation coefficient measures the strength and direction of association between two ranked variables.

```

Crime.Rate <- read.csv("/cloud/project/Data/Crime Rate.csv")
cor.test(Crime.Rate$X,Crime.Rate$Y, method ="pearson")

```

```

##
## Pearson's product-moment correlation
##
## data: Crime.Rate$X and Crime.Rate$Y
## t = -4.1029, df = 82, p-value = 9.571e-05
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.5761223 -0.2175580
## sample estimates:
## cor
## -0.4127033

```

```

cor.test(Crime.Rate$X,Crime.Rate$Y, method ="spearman")

```

```

## Warning in cor.test.default(Crime.Rate$X, Crime.Rate$Y, method = "spearman"):
## Cannot compute exact p-value with ties

```

```

##
## Spearman's rank correlation rho
##
## data: Crime.Rate$X and Crime.Rate$Y
## S = 140839, p-value = 5.359e-05

```

```
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## -0.4259324
```

P Value is < 0.05 \$. Reject H_o , $\rho \neq 0$. The correlation is not equal to zero.